

TRƯỜNG ĐẠI HỌC MỞ TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

LẬP TRÌNH CƠ SỞ DỮ LIỆU

ĐỀ TÀI:

HỆ THỐNG QUẢN LÝ PHÒNG KHÁM

Giảng viên hướng dẫn: ThS. Phạm Hoàng An

Nhóm thực hiện: Nhóm QLPK

Thành viên:

1. Châu Ngọc Sơn	– MSSV: 2351010184
2. Đỗ Phú Điền	– MSSV: 2351010048
3. Huỳnh Nhật Thiên Long	– MSSV: 2351010113

TP. Hồ Chí Minh, tháng 05 năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đề án môn *Lập trình cơ sở dữ liệu*, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn tận tình từ **ThS. Phạm Hoàng An** – giảng viên trực tiếp giảng dạy bộ môn. Thầy đã cung cấp kiến thức nền tảng về T-SQL, ADO.NET, LINQ, Entity Framework Core và ASP.NET Core Web API, đồng thời định hướng kiến trúc đa lớp cho đề án.

Nhóm cũng xin cảm ơn sự hợp tác chặt chẽ và tinh thần làm việc nghiêm túc giữa các thành viên, giúp đề án hoàn thành đúng thời hạn với chất lượng tốt nhất trong khả năng.

Do thời gian và kiến thức còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những góp ý quý báu của thầy để đề án ngày càng hoàn thiện hơn.

Nhóm thực hiện

Mục lục

Lời cảm ơn	1
1 GIỚI THIỆU CHUNG	6
1.1 Tên đề tài	6
1.2 Thông tin nhóm thực hiện	7
1.3 Mục tiêu đồ án	7
1.4 Đường link GitHub	8
2 PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU	9
2.1 Khảo sát nghiệp vụ	9
2.2 Sơ đồ thực thể mối quan hệ (ERD)	10
2.2.1 Mô tả chi tiết các bảng	10
2.2.2 Các ràng buộc dữ liệu	11
2.3 Lập trình phía máy chủ (T-SQL Scripts)	11
2.3.1 View truy vấn dữ liệu	12
2.3.2 User-defined Function	12
2.3.3 Stored Procedure	13
2.3.4 Quản lý Giao tác (Transaction)	13
2.3.5 Trigger	14
3 KIẾN TRÚC PHẦN MỀM	16
3.1 Mô hình 3 lớp + DTO	16
3.1.1 Presentation Layer – GUI_QLPK	17
3.1.2 Business Logic Layer – QLPKBUS	17
3.1.3 Data Access Layer – QLPKDAL	17
3.1.4 Tầng tích hợp Web API – GUI_QLPK/Services	17
3.1.5 Đối tượng trao đổi dữ liệu (DTO) – QLPKDTO	18
4 THỰC THI KỸ THUẬT VÀ CÔNG NGHỆ	20
4.1 ADO.NET	20
4.1.1 Mô hình kết nối (Connected)	20

4.1.2	Mô hình phi kết nối (Disconnected)	21
4.1.3	Gọi Function trên CSDL	21
4.2	LINQ	22
4.2.1	LINQ to Objects	22
4.2.2	LINQ to XML	23
4.3	Entity Framework Core 8	23
4.3.1	DbContext + DbSet	23
4.3.2	Code First – DataAnnotations + Fluent API	24
4.3.3	Database First – scaffold từ CSDL có sẵn	26
4.3.4	LINQ to Entities	26
4.4	ASP.NET Core 8 Web API	27
4.4.1	Cấu hình DI và Middleware	27
4.4.2	Controller chuẩn RESTful	28
4.4.3	Bảng các endpoint RESTful đã triển khai	29
4.4.4	Mẫu phản hồi JSON	29
4.4.5	Tiêu chuẩn RESTful	30
4.4.6	Swagger UI	30
5	KẾT QUẢ VÀ ĐÁNH GIÁ	31
5.1	Demo ứng dụng	31
5.1.1	Đăng nhập	31
5.1.2	Quản lý bệnh nhân	31
5.1.3	Tạo phiếu khám và kê toa thuốc	32
5.1.4	Lập hóa đơn và in PDF	32
5.1.5	Báo cáo doanh thu	32
5.2	Đánh giá chuẩn đầu ra	33
5.3	Khó khăn và hướng phát triển	33
5.3.1	Khó khăn	33
5.3.2	Hướng phát triển	34
	Tài liệu tham khảo	35

Danh sách hình vẽ

2.1	Sơ đồ ERD của hệ thống QLPK	10
3.1	Kiến trúc tổng thể hệ thống QLPK – 3 lớp + EF + Web API + DTO . .	16
4.1	Tài liệu API tự động bằng Swagger UI	30
5.1	Đăng nhập – kiểm tra qua <code>sp_KiemTraDangNhap</code>	31
5.2	Quản lý bệnh nhân – thêm/sửa/xóa mềm/tìm kiếm.	31
5.3	Kê toa – Trigger trừ kho tự động.	32
5.4	Lập hóa đơn – SP có Transaction, xuất PDF iText7.	32
5.5	Báo cáo doanh thu tháng (DataSet + DataAdapter + LINQ).	32

Danh sách bảng

1.1	Danh sách thành viên và phân công nhiệm vụ	7
2.1	Mô tả các bảng trong CSDL QLPK	11
2.2	Danh sách View trong CSDL	12
4.1	Danh sách 17 endpoint RESTful đã triển khai	29
5.1	Tự đánh giá mức độ đạt chuẩn đầu ra của môn học	33

1. GIỚI THIỆU CHUNG

1.1 Tên đề tài

Hệ thống Quản lý Phòng khám (QLPK) là ứng dụng nghiệp vụ cho phép quản trị viên, bác sĩ, điều dưỡng và nhân viên thu ngân của một cơ sở khám chữa bệnh cấp nhỏ và vừa thực hiện toàn bộ các nghiệp vụ chuyên môn: tiếp nhận bệnh nhân, đặt và theo dõi lịch hẹn, khám bệnh, kê toa thuốc, lập hóa đơn thanh toán, lập báo cáo doanh thu và báo cáo sử dụng thuốc.

1.2 Thông tin nhóm thực hiện

Bảng 1.1: Danh sách thành viên và phân công nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ
1	Châu Ngọc Sơn	2351010184	Phân tích yêu cầu, thiết kế CSDL, viết toàn bộ T-SQL Scripts (15 bảng, 13 View, 2 Function, 25 Stored Procedure, 4 Trigger). Quản lý Transaction trên CSDL.
2	Đỗ Phú Điền	2351010048	Lập trình lớp DAL (ADO.NET kết nối + phi kết nối với DataSet/DataAdapter), lớp DTO. Lập trình project QLPKEF (Entity Framework Core 8 – Code First + Database First).
3	Huỳnh Nhật Thiên Long	2351010113	Lập trình lớp BUS, áp dụng LINQ to Objects và LINQ to XML. Xây dựng project QLPKWebAPI (ASP.NET Core Web API RESTful + JSON). Lập trình giao diện Windows Forms (Guna UI), tích hợp báo cáo PDF, gửi mail nhắc lịch. Viết báo cáo LaTeX.

1.3 Mục tiêu đồ án

Đồ án nhằm vận dụng tổng hợp các kiến thức đã học để giải quyết một bài toán nghiệp vụ thực tế, cụ thể:

- Phân tích, thiết kế cơ sở dữ liệu quan hệ trên **Microsoft SQL Server**.
- Thành thạo **T-SQL**: View, Stored Procedure, User-defined Function, Trigger và quản lý Transaction.
- Xây dựng ứng dụng theo **kiến trúc 3 lớp** (Presentation – Business – Data Access) kết hợp lớp đối tượng truyền dữ liệu (DTO).
- Sử dụng **ADO.NET** cả mô hình *kết nối* (SqlConnection / SqlCommand / SqlDataReader) lẫn *phi kết nối* (SqlDataAdapter / DataSet / DataTable).

- Áp dụng **LINQ to Objects** và **LINQ to XML** trong tầng nghiệp vụ.
- Triển khai **Entity Framework Core 8** theo cả mô hình *Database First* và *Code First* với `DbContext` và `DbSet`.
- Xây dựng **ASP.NET Core 8 Web API** chuẩn **RESTful**, định dạng phản hồi **JSON**, có Swagger.

1.4 Đường link GitHub

Mã nguồn dự án

Repository (Public): <https://github.com/ThinkLonk/QuanLyPhongKham-WebAPI>

Nhánh chính: `main`

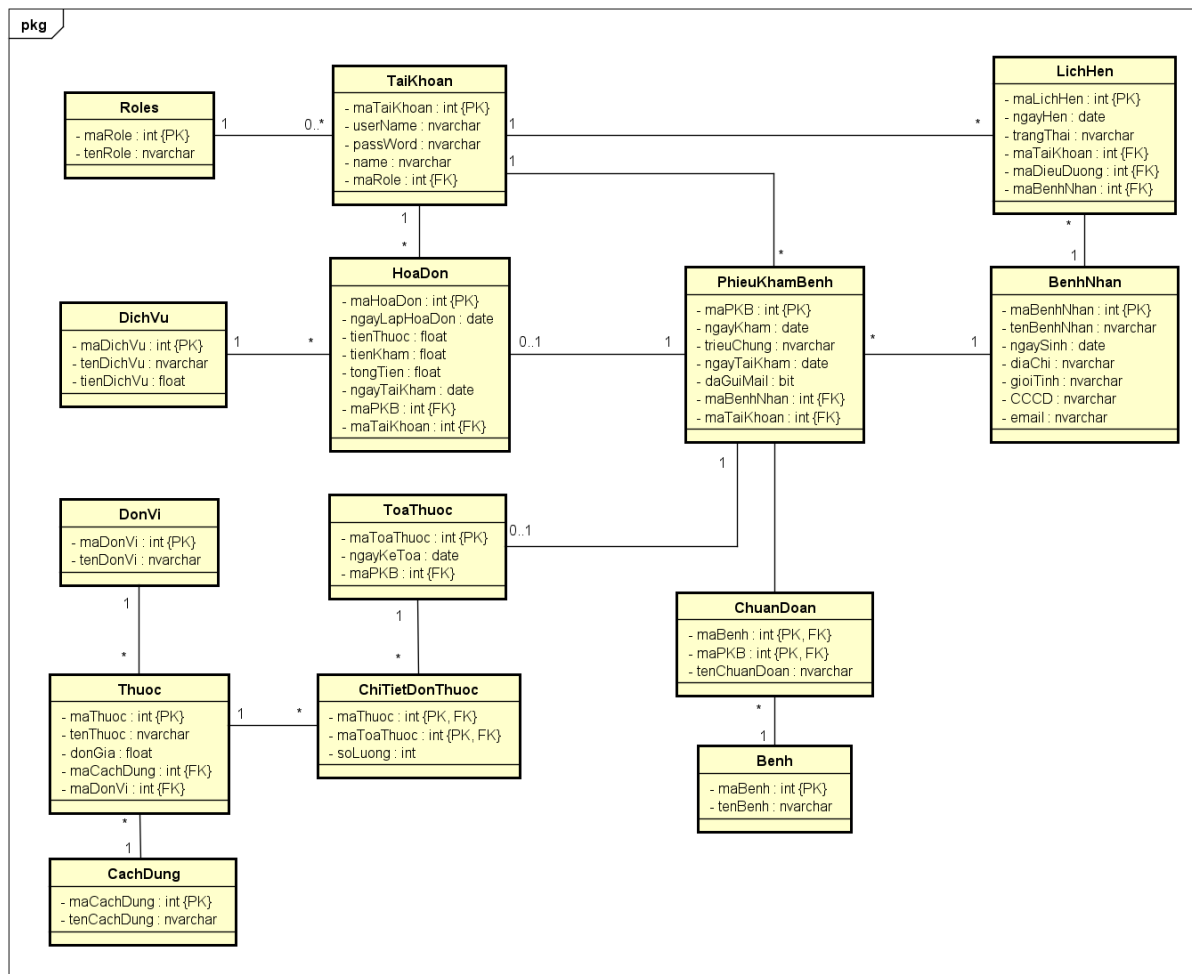
Hướng dẫn cài đặt: xem file `README.md` ở thư mục gốc của repo.

2. PHÂN TÍCH VÀ THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1 Khảo sát nghiệp vụ

Phòng khám hoạt động theo quy trình: bệnh nhân được điều dưỡng tiếp nhận và đặt lịch hẹn → đến ngày hẹn, bác sĩ tạo *Phiếu khám bệnh*, ghi triệu chứng và chẩn đoán → bác sĩ kê *Toa thuốc* gồm nhiều *Chi tiết đơn thuốc* (số lượng và đơn vị) → nhân viên thu ngân lập *Hóa đơn* trên cơ sở tiền thuốc (tính từ toa) và tiền khám (lấy từ *Dịch vụ*). Hệ thống lưu lịch sử khám của bệnh nhân, sinh báo cáo doanh thu theo ngày/tháng, báo cáo sử dụng thuốc theo tháng.

2.2 Sơ đồ thực thể mối quan hệ (ERD)



Hình 2.1: Sơ đồ ERD của hệ thống QLPK

CSDL gồm 15 bảng được phân nhóm như sau:

- **Nhóm tài khoản & phân quyền:** Roles, TaiKhoan, AuditLog.
- **Nhóm bệnh nhân & khám:** BenhNhan, LichHen, PhieuKhamBenh, Benh, ChuanDoan.
- **Nhóm thuốc & toa:** Thuoc, DonVi, CachDung, ToaThuoc, ChiTietDonThuoc.
- **Nhóm dịch vụ & hóa đơn:** DichVu, HoaDon.

2.2.1 Mô tả chi tiết các bảng

Bảng	Khóa chính	Mô tả
TaiKhoan	maTaiKhoan	Tài khoản người dùng. Ràng buộc UNIQUE trên <code>userName</code> .
Roles	maRole	Vai trò: Bác sĩ, Thu ngân, Điều dưỡng, Quản trị viên (theo dữ liệu thực tế của bảng <code>Roles</code>).
BenhNhan	maBenhNhan	Hồ sơ bệnh nhân. Có cờ <code>isDeleted</code> cho <i>soft-delete</i> , UNIQUE trên CCCD.
LichHen	maLichHen	Lịch hẹn khám. Trạng thái mặc định <i>Chờ khám</i> .
PhieuKhamBenh	maPKB	Phiếu khám: ngày khám, triệu chứng, ngày tái khám, cờ đã gửi mail.
Benh, ChuanDoan	maBenh / khóa kép	Danh mục bệnh và chẩn đoán cụ thể.
Thuoc	maThuoc	Danh mục thuốc: đơn giá, đơn vị, cách dùng, tồn kho.
DonVi, CachDung	maDonVi / maCachDung	Danh mục đơn vị, cách dùng thuốc.
ToaThuoc	maToaThuoc	Toa thuốc gắn với phiếu khám.
ChiTietDonThuoc	khóa kép	Bảng nối nhiều-nhiều giữa toa và thuốc, có <code>soLuong</code> .
DichVu	maDichVu	Bảng giá dịch vụ (Khám bệnh, Tiểu phẫu...).
HoaDon	maHoaDon	Hóa đơn: tiền thuốc, tiền khám, tổng tiền.
AuditLog	logID	Nhật ký thao tác phục vụ kiểm toán.

Bảng 2.1: Mô tả các bảng trong CSDL QLPK

2.2.2 Các ràng buộc dữ liệu

- 17 ràng buộc khóa ngoại (FOREIGN KEY).
- 3 CHECK CONSTRAINT: `chk_SoLuongKe` (> 0), `chk_SoLuongThuoc` (≥ 0), `chk_TienDichVu` (≥ 0).
- Giá trị mặc định cho các cột thời gian (`GETDATE()`) và cờ trạng thái.

2.3 Lập trình phía máy chủ (T-SQL Scripts)

2.3.1 View truy vấn dữ liệu

Toàn bộ thao tác đọc của ứng dụng đều đi qua **13 View** nhằm tách biệt cấu trúc bảng vật lý khỏi tầng ứng dụng:

View
v_DanhSachBenhNhan
v_DanhSachTaiKhoan
v_DanhSachLichHen
v_DanhSachThuoc
v_DanhSachDichVu, v_DanhSachBenh, v_DanhSachDonVi, v_DanhSachCachDung, v_DanhSachRoles, v_DanhSach
v_ChiTietPhieuKham
v_ChiTietDonThuoc
v_DanhSachHoaDon

Bảng 2.2: Danh sách View trong CSD

```

1 CREATE VIEW dbo.v_DanhSachThuoc AS
2 SELECT  t.maThuoc, t.tenThuoc, t.donGia, t.soLuong,
3         t.maDonVi, t.maCachDung,
4         dv.tenDonVi, cd.tenCachDung
5 FROM    Thuoc t
6 JOIN    DonVi dv      ON t.maDonVi      = dv.maDonVi
7 JOIN    CachDung cd   ON t.maCachDung = cd.maCachDung;

```

Listing 2.1: View v_DanhSachThuoc – chuẩn hóa truy vấn danh mục thuốc

2.3.2 User-defined Function

(1) **Scalar Function** f_TinhTienThuoc tính tổng tiền thuốc của một Phiếu khám:

```

1 CREATE FUNCTION dbo.f_TinhTienThuoc(@maPKB INT)
2 RETURNS FLOAT
3 AS
4 BEGIN
5     DECLARE @TongTien FLOAT;
6     SELECT @TongTien = SUM(th.donGia * ctdt.soLuong)
7     FROM    ToaThuoc tt
8     JOIN    ChiTietDonThuoc ctdt ON tt.maToaThuoc = ctdt.maToaThuoc
9     JOIN    Thuoc th           ON ctdt.maThuoc   = th.maThuoc
10    WHERE   tt.maPKB = @maPKB;
11    RETURN ISNULL(@TongTien, 0);

```

12 **END;**

Listing 2.2: Scalar Function tính tiền thuốc

(2) **Table-Valued Function** `f_LichSuKhamBenh` trả về lịch sử khám của một bệnh nhân:

```

1 CREATE FUNCTION dbo.f_LichSuKhamBenh(@maBenhNhan INT)
2 RETURNS TABLE
3 AS RETURN
4 (
5     SELECT pkb.maPKB, pkb.ngayKham, pkb.trieuChung,
6           STRING_AGG(th.tenThuoc, ', ') AS ToaThuoc,
7           (dbo.f_TinhTienThuoc(pkb.maPKB) + ISNULL(hd.tienKham,0)) AS
            TongChiPhi
8 FROM     PhieuKhamBenh pkb
9 LEFT JOIN ToaThuoc tt          ON pkb.maPKB      = tt.maPKB
10 LEFT JOIN ChiTietDonThuoc ctdt ON tt.maToaThuoc = ctdt.maToaThuoc
11 LEFT JOIN Thuoc th            ON ctdt.maThuoc  = th.maThuoc
12 LEFT JOIN HoaDon hd          ON pkb.maPKB      = hd.maPKB
13 WHERE    pkb.maBenhNhan = @maBenhNhan
14 GROUP BY pkb.maPKB, pkb.ngayKham, pkb.trieuChung, hd.tienKham
15 );

```

Listing 2.3: Table-Valued Function tra cứu lịch sử khám

2.3.3 Stored Procedure

Toàn bộ thao tác Insert/Update/Delete đều đóng gói trong **25 Stored Procedure** để chống SQL Injection. Các nhóm chính: đăng nhập & tài khoản, bệnh nhân, bệnh, thuốc, lịch hẹn, phiếu khám (có Transaction), hóa đơn (có Transaction), dịch vụ, báo cáo.

2.3.4 Quản lý Giao tác (Transaction)

Hai nghiệp vụ *Tạo phiếu khám + toa thuốc* và *Lập hóa đơn* được bao trong `BEGIN TRANSACTION ... COMMIT/ROLLBACK` kèm `TRY-CATCH`:

```

1 CREATE PROCEDURE dbo.sp_LapHoaDonThanhToan
2     @maPKB INT, @maTaiKhoan INT
3 AS
4 BEGIN
5     SET NOCOUNT ON;
6     BEGIN TRANSACTION;
7     BEGIN TRY

```

```

8      DECLARE @tienThuoc FLOAT = dbo.f_TinhTienThuoc(@maPKB);
9      DECLARE @tienKham FLOAT;
10     SELECT @tienKham = tienDichVu FROM DichVu WHERE tenDichVu = N'Khám
        ệbnh';
11     IF @tienKham IS NULL SET @tienKham = 50000;
12
13     DECLARE @tongTien FLOAT = @tienThuoc + @tienKham;
14     DECLARE @ngayTaiKham DATE;
15     SELECT @ngayTaiKham = ngayTaiKham FROM PhieuKhamBenh WHERE maPKB =
        @maPKB;
16
17     INSERT INTO HoaDon(ngayLapHoaDon, tienThuoc, tienKham, tongTien,
18                       ngayTaiKham, maPKB, maTaiKhoan)
19     VALUES (GETDATE(), @tienThuoc, @tienKham, @tongTien,
20            @ngayTaiKham, @maPKB, @maTaiKhoan);
21     COMMIT TRANSACTION;
22 END TRY
23 BEGIN CATCH
24     ROLLBACK TRANSACTION;
25     DECLARE @msg NVARCHAR(4000) = ERROR_MESSAGE();
26     RAISERROR(@msg, 16, 1);
27 END CATCH
28 END;

```

Listing 2.4: SP sp_LapHoaDonThanhToan – sử dụng Transaction

2.3.5 Trigger

Hệ thống cài đặt 4 **Trigger**:

1. trg_CapNhatSoLuongThuocTrongKho – AFTER INSERT: kiểm tra tồn kho và tự động trừ kho khi kê thuốc.
2. trg_HoanTraSoLuongThuoc – AFTER DELETE: hoàn trả tồn kho khi xóa thuốc khỏi đơn.
3. trg_KiemTraNgayDatLich – AFTER INSERT, UPDATE: chặn đặt lịch trong quá khứ.
4. trg_ChanXoaTaiKhoan – INSTEAD OF DELETE: không cho xóa tài khoản có lịch sử.

```

1 CREATE TRIGGER trg_CapNhatSoLuongThuocTrongKho
2 ON ChiTietDonThuoc
3 AFTER INSERT
4 AS

```

```
5 BEGIN
6     IF EXISTS (SELECT 1 FROM inserted i
7                 JOIN Thuoc t ON i.maThuoc = t.maThuoc
8                 WHERE t.soLuong < i.soLuong)
9     BEGIN
10        RAISERROR(N'Không đủ s ượng ốthuc trong kho!', 16, 1);
11        ROLLBACK TRANSACTION;
12        RETURN;
13    END
14    UPDATE t SET t.soLuong = t.soLuong - i.soLuong
15    FROM Thuoc t JOIN inserted i ON t.maThuoc = i.maThuoc;
16 END;
```

Listing 2.5: Trigger kiểm tra & cập nhật tồn kho thuốc

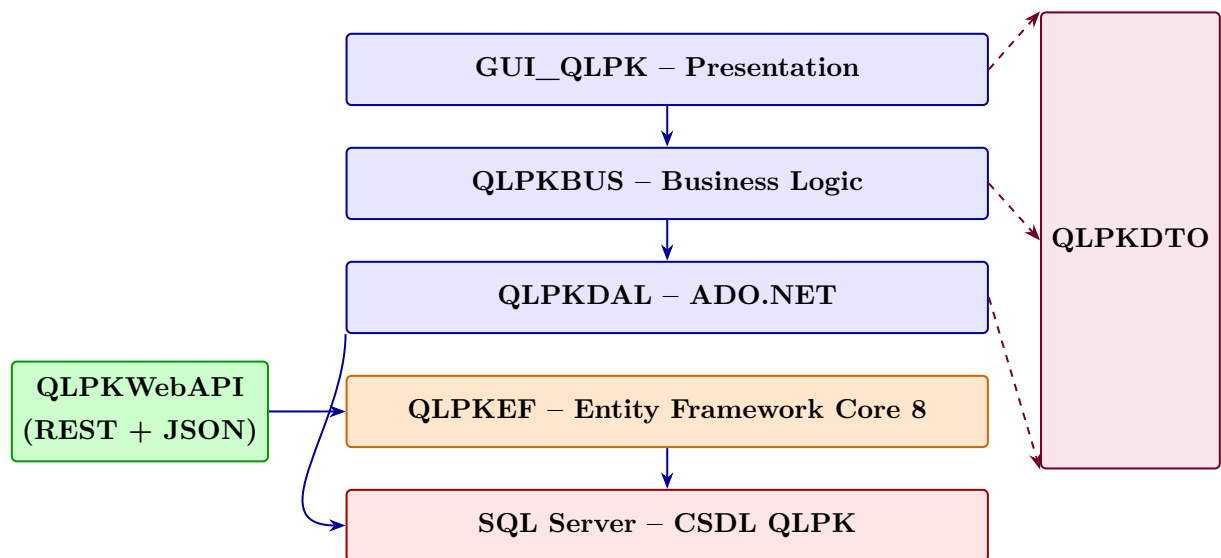
3. KIẾN TRÚC PHẦN MỀM

3.1 Mô hình 3 lớp + DTO

Ứng dụng được tổ chức thành **6 project** trong cùng solution `QuanLyPhongKham.sln`:

- GUI_QLPK – Presentation Layer (WinForms, Guna UI).
- QLPKBUS – Business Logic Layer.
- QLPKDAL – Data Access Layer (ADO.NET).
- QLPKDTO – Data Transfer Object.
- QLPKEF – Entity Framework Core 8 (Code First + Database First).
- QLPKWebAPI – ASP.NET Core 8 Web API (RESTful + JSON).

Ngoài ra, project GUI_QLPK có thêm thư mục `Services` đóng vai trò *API Client* (`ApiClient`, `AuthService`, `BenhNhanService`, `HoaDonService`, `LichHenService`, `ThuocService`) để gọi Web API qua `HttpClient` với cơ chế *fallback* về tầng BUS/DAL khi API ngoại tuyến.



Hình 3.1: Kiến trúc tổng thể hệ thống QLPK – 3 lớp + EF + Web API + DTO

3.1.1 Presentation Layer – GUI_QLPK

Cung cấp các Form: Login, QLPMMain, Home, DangKyKham, DSLaySoKham, QuanLyBenhNhan, ThemPhieuKhamBenh, KeToa, LapHoaDon, DanhSachHoaDon, BaoCaoDoanhThu, BaoCaoSuDungThuoc, QuanLyTaiKhoan, QuanLyThuoc, QuanLyDichVu, QuanLyLoaiBenh, QuanLyNhacHen, TraCuuBenhNhan. Sử dụng **Guna.UI2.WinForms** cho giao diện hiện đại, **iText7** xuất hóa đơn PDF, **SMTP** gửi mail nhắc tái khám.

3.1.2 Business Logic Layer – QLPKBUS

Mỗi entity có một class BUS: BenhNhanBUS, HoadonBUS, PhieukhambenhBUS, ThuocBUS... Vai trò: kiểm tra ràng buộc, áp dụng LINQ to Objects, đóng vai trò *facade* cho lớp GUI.

```

1 namespace QLPKBUS
2 {
3     public class BenhNhanBUS
4     {
5         private BenhNhanDAL bnDAL;
6         public BenhNhanBUS() { bnDAL = new BenhNhanDAL(); }
7         public bool them (BenhNhanDTO bn) => bnDAL.Them(bn);
8         public bool sua (BenhNhanDTO bn, int oldId) => bnDAL.Sua(bn,
9             oldId);
10        public bool xoa (BenhNhanDTO bn) => bnDAL.Xoa(bn);
11        public List<BenhNhanDTO> select() => bnDAL.select();
12        public List<BenhNhanDTO> selectByKeyWord(string kw) =>
13            bnDAL.SelectByKeyWord(kw);
14    }
15 }
```

Listing 3.1: Lớp BUS trung gian giữa GUI và DAL

3.1.3 Data Access Layer – QLPKDAL

14 class DAL (benhnhanDAL, HoadonDAL, PhieukhambenhDAL...) sử dụng **ADO.NET** với **SqlConnection**, **SqlCommand**, **SqlDataReader**, **SqlParameter**; gọi Stored Procedure hoặc View. Chuỗi kết nối lấy từ App.config.

3.1.4 Tầng tích hợp Web API – GUI_QLPK/Services

Để minh chứng cho việc tích hợp Web API ở cả phía client, project GUI_QLPK bổ sung thư mục Services gồm ApiClient, ApiContracts, AuthService, BenhNhanService, HoaDonService, LichHenService, ThuocService. Mỗi service ưu tiên gọi Web API (qua HttpClient + Newtonsoft.Json);

nếu Web API không khả dụng (tắt cờ `ApiEnabled` trong `App.config` hoặc lỗi mạng) sẽ **fallback** sang tầng BUS/DAL truyền thống. Nhờ đó WinForms client *không phụ thuộc* vào Web API mà vẫn chạy độc lập, đồng thời chứng minh kiến trúc đã thực sự tách lớp.

```

1 internal class AuthService
2 {
3     private readonly taiKhoanBUS fallback = new taiKhoanBUS();
4
5     public bool TryLogin(string username, string password, out int maTaiKhoan)
6     {
7         maTaiKhoan = 0;
8         var request = new ApiLoginRequest { UserName = username, PassWord =
          password };
9
10        // 1) Gọi Web API (REST + JSON)
11        if (ApiClient.TryPost("api/Auth/login", request, out ApiLoginResponse
          response)
12            && response != null)
13        {
14            maTaiKhoan = response.MaTaiKhoan;
15            return true;
16        }
17
18        // 2) Fallback dùng BUS/DAL (ADO.NET) nếu API offline
19        foreach (var tk in fallback.select())
20            if (tk.Username == username && tk.Password == password)
21                { maTaiKhoan = tk.MaTK; return true; }
22
23        return false;
24    }
25 }

```

Listing 3.2: GUI_QLPK/Services/AuthService.cs – gọi Web API có fallback

3.1.5 Đối tượng trao đổi dữ liệu (DTO) – QLPKDTO

Project DTO chứa các lớp POCO độc lập với ADO.NET/WinForms, được chia sẻ giữa các lớp:

```

1 namespace QLPKDTO
2 {
3     public class BenhNhanDTO

```

```
4      {  
5          public int      MaBN          { get; set; }  
6          public string   TenBN         { get; set; }  
7          public string   DiachiBN     { get; set; }  
8          public string   GtBN         { get; set; }  
9          public DateTime NgsinhBN     { get; set; }  
10         public string   CanCuocCongDan { get; set; }  
11         public string   Email        { get; set; }  
12     }  
13 }
```

Listing 3.3: DTO – BenhNhanDTO

4. THỰC THI KỸ THUẬT VÀ CÔNG NGHỆ

4.1 ADO.NET

4.1.1 Mô hình kết nối (Connected)

Mọi thao tác đọc/ghi trong các Form chính sử dụng `SqlConnection`, `SqlCommand`, `SqlDataReader` với **Parameterized Query** chống SQL Injection:

```
1 <appSettings>
2   <add key="ConnectionString"
3       value="Data Source=localhost;Initial Catalog=QLPK;Integrated
4       Security=True"/>
</appSettings>
```

Listing 4.1: Cấu hình chuỗi kết nối trong App.config

```
1 public List<BenhNhanDTO> select()
2 {
3     var ds = new List<BenhNhanDTO>();
4     using (var con = new SqlConnection(ConnectionString))
5     using (var cmd = new SqlCommand("SELECT * FROM v_DanhSachBenhNhan", con))
6     {
7         cmd.CommandType = CommandType.Text;
8         con.Open();
9         using (var rd = cmd.ExecuteReader())
10        {
11            while (rd.Read())
12            {
13                ds.Add(new BenhNhanDTO {
14                    MaBN      = (int)rd["maBenhNhan"],
15                    TenBN      = rd["tenBenhNhan"].ToString(),
16                    GtBN       = rd["gioiTinh"].ToString(),
17                    NgsinhBN   = (DateTime)rd["ngaySinh"],
```

```

18         DiachiBN = rd["diaChi"].ToString(),
19         CanCuocCongDan = rd["CCCD"].ToString(),
20         Email      = rd["email"] == DBNull.Value ? "" :
                rd["email"].ToString()
21     });
22 }
23 }
24 }
25 return ds;
26 }

```

Listing 4.2: DAL gọi View qua ADO.NET (mô hình kết nối)

4.1.2 Mô hình phi kết nối (Disconnected)

Các Form báo cáo (BaoCaoDoanhThu, BaoCaoSuDungThuoc) sử dụng SqlDataAdapter + DataSet / DataTable để load toàn bộ kết quả về RAM, sau đó hiển thị trên DataGridView và xuất PDF. Đây là mô hình *phi kết nối* – connection được đóng ngay sau khi Fill:

```

1 public DataSet LayBaoCaoSuDungThuoc(int thang, int nam)
2 {
3     var ds = new DataSet();
4     using (var con = new SqlConnection(ConnectionString))
5     using (var cmd = new SqlCommand("sp_BaoCaoSuDungThuoc", con))
6     {
7         cmd.CommandType = CommandType.StoredProcedure;
8         cmd.Parameters.AddWithValue("@month", thang);
9         cmd.Parameters.AddWithValue("@year", nam);
10
11         using (var adapter = new SqlDataAdapter(cmd))
12         {
13             adapter.Fill(ds, "BaoCaoThuoc"); // Connection ởm -> đọc -> đóng
14             // ặt động
15         }
16     }
17     return ds; // ảTr DataSet về để binding vào DataGridView
18 }

```

Listing 4.3: Mô hình phi kết nối – SqlDataAdapter + DataSet

4.1.3 Gọi Function trên CSDL

```

1 public decimal tienthuoc(hoadonDTO hd, int maPKB)
2 {
3     using (var con = new SqlConnection(ConnectionString))
4     using (var cmd = new SqlCommand("SELECT dbo.f_TinhTienThuoc(@maPKB)",
5         con))
6     {
7         cmd.Parameters.AddWithValue("@maPKB", maPKB);
8         con.Open();
9         var result = cmd.ExecuteScalar();
10        return result == DBNull.Value ? 0m : Convert.ToDecimal(result);
11    }
12 }

```

Listing 4.4: Gọi Scalar Function f_TinhTienThuoc qua ADO.NET

4.2 LINQ

4.2.1 LINQ to Objects

Áp dụng tại tầng BUS và GUI để xử lý các danh sách DTO – lọc, sắp xếp, gom nhóm doanh thu theo ngày, đếm số bệnh nhân theo bác sĩ:

```

1 var doanhThuTheoNgay = lsHoaDon
2     .Where(h => h.NgayLapHoaDon.Month == thang && h.NgayLapHoaDon.Year == nam)
3     .GroupBy(h => h.NgayLapHoaDon.Date)
4     .Select(g => new {
5         Ngay      = g.Key,
6         TongTien  = g.Sum(x => x.TongTien),
7         SoHoaDon  = g.Count()
8     })
9     .OrderBy(x => x.Ngay)
10    .ToList();

```

Listing 4.5: LINQ to Objects – gom nhóm hóa đơn theo ngày

Một ví dụ khác trong project QLPKEF:

```

1 public static IEnumerable<Thuoc> LocThuocSapHet(IEnumerable<Thuoc> nguon, int
2     nguong = 10) =>
3     nguon.Where(t => (t.SoLuong ?? 0) <= nguong)
4         .OrderBy(t => t.SoLuong)
5         .ThenBy(t => t.TenThuoc);

```

Listing 4.6: LINQ to Objects – lọc thuốc tồn kho thấp

4.2.2 LINQ to XML

Đã triển khai chức năng **xuất / nhập danh mục thuốc dạng XML** (file QLPKEF/Samples/LinqToXmlExport.cs) để trao đổi dữ liệu với phần mềm bảo hiểm y tế:

```

1 public static async Task ExportDanhMucThuocAsync(QLPKDbContext db, string
   filePath)
2 {
3     var thuocs = await db.Thuocs
4         .Include(t => t.DonVi)
5         .Include(t => t.CachDung)
6         .ToListAsync();
7
8     var xdoc = new XDocument(
9         new XDeclaration("1.0", "utf-8", "yes"),
10        new XComment("Danh mục thuốc xuất ết hệ thống QLPK"),
11        new XElement("DanhSachThuoc",
12            new XAttribute("ngayXuat", DateTime.Now.ToString("yyyy-MM-dd")),
13            from t in thuocs
14            select new XElement("Thuoc",
15                new XAttribute("ma", t.MaThuoc),
16                new XElement("Ten", t.TenThuoc),
17                new XElement("DonGia", t.DonGia),
18                new XElement("DonVi", t.DonVi?.TenDonVi),
19                new XElement("CachDung", t.CachDung?.TenCachDung),
20                new XElement("SoLuong", t.SoLuong ?? 0))));
21    xdoc.Save(filePath);
22 }

```

Listing 4.7: LINQ to XML – xuất danh mục thuốc ra file XML

4.3 Entity Framework Core 8

Đồ án đã triển khai project riêng QLPKEF sử dụng **Entity Framework Core 8.0.10** trên nền **.NET 8**, hỗ trợ cả **Code First** lẫn **Database First**.

4.3.1 DbContext + DbSet

```

1 public class QLPKDbContext : DbContext
2 {
3     public DbSet<TaiKhoan> TaiKhoans => Set<TaiKhoan>();
4     public DbSet<Role> Roles => Set<Role>();
5     public DbSet<BenhNhan> BenhNhans => Set<BenhNhan>();
6     public DbSet<LichHen> LichHens => Set<LichHen>();
7     public DbSet<PhieuKhamBenh> PhieuKhamBenhs =>
        Set<PhieuKhamBenh>();
8     public DbSet<Benh> Benhs => Set<Benh>();
9     public DbSet<ChuanDoan> ChuanDoans => Set<ChuanDoan>();
10    public DbSet<Thuoc> Thuocs => Set<Thuoc>();
11    public DbSet<DonVi> DonVis => Set<DonVi>();
12    public DbSet<CachDung> CachDungs => Set<CachDung>();
13    public DbSet<ToaThuoc> ToaThuocs => Set<ToaThuoc>();
14    public DbSet<ChiTietDonThuoc> ChiTietDonThuocs =>
        Set<ChiTietDonThuoc>();
15    public DbSet<HoaDon> HoaDons => Set<HoaDon>();
16    public DbSet<DichVu> DichVus => Set<DichVu>();
17
18    public QLPKDbContext(DbContextOptions<QLPKDbContext> options) :
        base(options) { }
19 }

```

Listing 4.8: QLPKDbContext – 14 DbSet ứng với 14 bảng nghiệp vụ

4.3.2 Code First – DataAnnotations + Fluent API

Mỗi entity được cấu hình bằng DataAnnotations:

```

1 [Table("BenhNhan")]
2 public class BenhNhan
3 {
4     [Key]
5     [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
6     public int MaBenhNhan { get; set; }
7
8     [Required, MaxLength(50)] public string TenBenhNhan { get; set; } = null!;
9     public DateTime NgaySinh { get; set; }
10    [Required, MaxLength(50)] public string DiaChi { get; set; } = null!;
11    [Required, MaxLength(12)] public string CCCD { get; set; } = null!;
12    [Required, MaxLength(50)] public string GioiTinh { get; set; } = null!;
13    [MaxLength(100)] public string? Email { get; set; }

```

```

14     public bool IsDeleted { get; set; }
15 }

```

Listing 4.9: Entity BenhNhan – Code First DataAnnotations

Các ràng buộc phức tạp (composite key, unique index, check constraint, cascade) được khai báo qua Fluent API trong `OnModelCreating`:

```

1  protected override void OnModelCreating(ModelBuilder modelBuilder)
2  {
3      // Composite key
4      modelBuilder.Entity<ChuanDoan>().HasKey(c => new { c.MaBenh, c.MaPKB });
5      modelBuilder.Entity<ChiTietDonThuoc>().HasKey(c => new { c.MaThuoc,
6          c.MaToaThuoc });
7
8      // Unique constraints
9      modelBuilder.Entity<BenhNhan>().HasIndex(b => b.CCCD).IsUnique();
10     modelBuilder.Entity<TaiKhoan>().HasIndex(t => t.UserName).IsUnique();
11     modelBuilder.Entity<Thuoc>().HasIndex(t => t.TenThuoc).IsUnique();
12
13     // CHECK constraints (EF Core 7+ syntax)
14     modelBuilder.Entity<Thuoc>()
15         .ToTable(t => t.HasCheckConstraint("chk_SoLuongThuoc", "[soLuong] >=
16             0"));
17     modelBuilder.Entity<DichVu>()
18         .ToTable(t => t.HasCheckConstraint("chk_TienDichVu", "[tienDichVu]
19             >= 0"));
20     modelBuilder.Entity<ChiTietDonThuoc>()
21         .ToTable(t => t.HasCheckConstraint("chk_SoLuongKe", "[soLuong] >
22             0"));
23
24     // Default values
25     modelBuilder.Entity<LichHen>().Property(l => l.TrangThai)
26         .HasDefaultValue("ởCh khám");
27     modelBuilder.Entity<PhieuKhamBenh>().Property(p => p.NgayKham)
28         .HasDefaultValueSql("(getdate())");
29
30     // Tránh cascade cycle
31     modelBuilder.Entity<PhieuKhamBenh>().HasOne(p => p.TaiKhoan).WithMany(t
32         => t.PhieuKham)
33         .HasForeignKey(p => p.MaTaiKhoan).OnDelete(DeleteBehavior.NoAction);
34 }

```

Listing 4.10: Fluent API trong OnModelCreating

Tạo CSDL từ entity (lệnh Code First migration):

```
1 dotnet ef migrations add InitialCreate --project QLPKEF
2 dotnet ef database update --project QLPKEF
```

Listing 4.11: Lệnh Code First

4.3.3 Database First – scaffold từ CSDL có sẵn

```
1 dotnet ef dbcontext scaffold ^
2     "Server=localhost;Database=QLPK;Trusted_Connection=True;TrustServerCertificate=True"
3     ^
4     Microsoft.EntityFrameworkCore.SqlServer ^
5     --output-dir Models --context QLPKDbContext --force
```

Listing 4.12: Sinh entity từ CSDL bằng Database First

4.3.4 LINQ to Entities

Các Repository (BenhNhanRepository, HoaDonRepository) thao tác CSDL hoàn toàn qua LINQ:

```
1 public async Task<List<BenhNhan>> GetAllAsync() =>
2     await _db.BenhNhans.AsNoTracking()
3         .Where(b => !b.IsDeleted)
4         .OrderBy(b => b.TenBenhNhan)
5         .ToListAsync();
6
7 public async Task<List<DoanhThuTheoNgayDto>> DoanhThuTheoThangAsync(int
8     thang, int nam) =>
9     await _db.HoaDons
10        .Where(h => h.NgayLapHoaDon!.Value.Month == thang
11            && h.NgayLapHoaDon.Value.Year == nam)
12        .GroupBy(h => h.NgayLapHoaDon!.Value.Date)
13        .Select(g => new DoanhThuTheoNgayDto {
14            Ngay = g.Key,
15            SoHoaDon = g.Count(),
16            TongTien = g.Sum(x => x.TongTien ?? 0)
17        })
18        .OrderBy(x => x.Ngay)
19        .ToListAsync();
20 // ọGi Stored Procedure đã có sẵn qua EF Core
```

```
21 public async Task<int> LapHoaDonAsync(int maPKB, int maTaiKhoan) =>
22     await _db.Database.ExecuteSqlInterpolatedAsync(
23         $"EXEC sp_LapHoaDonThanhToan @maPKB={maPKB},
           @maTaiKhoan={maTaiKhoan}");
```

Listing 4.13: Repository – LINQ to Entities trên DbContext

4.4 ASP.NET Core 8 Web API

Đồ án đã triển khai project QLPKWebAPI sử dụng **ASP.NET Core 8**, tuân thủ chuẩn **RESTful**, phản hồi định dạng **JSON**, tích hợp **Swagger UI** cho tài liệu API.

4.4.1 Cấu hình DI và Middleware

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 // EF Core + SQL Server
4 builder.Services.AddDbContext<QLPKDbContext>(opt =>
5     opt.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
6
7 // DI cho repositories
8 builder.Services.AddScoped<BenhNhanRepository>();
9 builder.Services.AddScoped<HoaDonRepository>();
10
11 // Hỗ trợ các JSON và XML
12 builder.Services.AddControllers()
13     .AddXmlSerializerFormatters();
14
15 // Swagger
16 builder.Services.AddEndpointsApiExplorer();
17 builder.Services.AddSwaggerGen(c =>
18 {
19     c.SwaggerDoc("v1", new() {
20         Title = "QLPK Web API",
21         Version = "v1",
22         Description = "Web API quản lý phòng khám - EF Core + RESTful + JSON"
23     });
24 });
25
26 // CORS mở cho dev
27 builder.Services.AddCors(o => o.AddPolicy("Open",
```

```

28     p => p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod());
29
30 var app = builder.Build();
31 if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
32 app.UseCors("Open");
33 app.MapControllers();
34 app.Run();

```

Listing 4.14: Program.cs – cấu hình ASP.NET Core 8 Web API

4.4.2 Controller chuẩn RESTful

```

1 [ApiController]
2 [Route("api/[controller]")]
3 [Produces("application/json")]
4 public class BenhNhanController : ControllerBase
5 {
6     private readonly BenhNhanRepository _repo;
7     public BenhNhanController(BenhNhanRepository repo) => _repo = repo;
8
9     [HttpGet] // GET /api/benhnhhan
10    public async Task<ActionResult<IEnumerable<BenhNhan>>> GetAll()
11        => Ok(await _repo.GetAllAsync());
12
13    [HttpGet("search")] // GET /api/benhnhhan/search?keyword=
14    public async Task<ActionResult<IEnumerable<BenhNhan>>> Search([FromQuery]
15        string keyword)
16        => Ok(await _repo.SearchAsync(keyword ?? string.Empty));
17
18    [HttpGet("{id:int}/lichsu")] // GET /api/benhnhhan/{id}/lichsu
19    public async Task<ActionResult<IEnumerable<object>>> LichSu(int id)
20        => Ok(await _repo.GetLichSuKhamAsync(id));
21
22    [HttpPost] // POST /api/benhnhhan
23    public async Task<ActionResult<BenhNhan>> Create([FromBody] BenhNhan dto)
24    {
25        if (!ModelState.IsValid) return BadRequest(ModelState);
26        var created = await _repo.AddAsync(dto);
27        return CreatedAtAction(nameof(GetAll), new { id = created.MaBenhNhan
28        }, created);
29    }

```

```

29 [HttpPut("{id:int}")] // PUT /api/benhnhan/{id}
30 public async Task<IActionResult> Update(int id, [FromBody] BenhNhan dto)
31 {
32     if (id != dto.MaBenhNhan) return BadRequest("Mã không trùng ở khp");
33     var ok = await _repo.UpdateAsync(dto);
34     return ok ? NoContent() : NotFound();
35 }
36
37 [HttpDelete("{id:int}")] // DELETE /api/benhnhan/{id}
38 public async Task<IActionResult> Delete(int id)
39     => (await _repo.SoftDeleteAsync(id)) ? NoContent() : NotFound();
40 }

```

Listing 4.15: BenhNhanController – chuẩn REST CRUD

4.4.3 Bảng các endpoint RESTful đã triển khai

Method	Endpoint	Mô tả
POST	/api/auth/login	Đăng nhập (LINQ to Entities).
GET	/api/benhnhan	Danh sách bệnh nhân chưa xóa.
GET	/api/benhnhan/search?keyword=	Tìm kiếm theo tên hoặc CCCD.
GET	/api/benhnhan/{id}/lichsu	Lịch sử khám (JOIN nhiều bảng).
POST	/api/benhnhan	Thêm mới.
PUT	/api/benhnhan/{id}	Cập nhật.
DELETE	/api/benhnhan/{id}	Xóa mềm.
GET	/api/thuoc	Danh sách thuốc + đơn vị + cách dùng.
GET	/api/thuoc/saphet?nguon=10	Cảnh báo tồn kho thấp (LINQ to Objects).
GET	/api/thuoc/export-xml	Xuất XML (LINQ to XML).
POST	/api/thuoc	Thêm thuốc.
GET	/api/lichhen	Danh sách lịch hẹn.
POST	/api/lichhen	Tạo lịch hẹn (validate ngày).
PUT	/api/lichhen/{id}/trangthai	Cập nhật trạng thái.
GET	/api/hoadon	Toàn bộ hóa đơn.
GET	/api/hoadon/doanhthu?thang=&nam=	Báo cáo doanh thu theo tháng.
POST	/api/hoadon/lap	Lập hóa đơn (gọi SP có Transaction).

Bảng 4.1: Danh sách 17 endpoint RESTful đã triển khai

4.4.4 Mẫu phản hồi JSON

```
1 {  
2   "maTaiKhoan": 1,  
3   "userName":   "admin",  
4   "name":       "Quan tri vien",  
5   "tenRole":    "Quan tri"  
6 }
```

Listing 4.16: Response mẫu của POST /api/auth/login

```
1 [  
2   { "ngay": "2026-05-02", "soHoaDon": 12, "tongTien": 4350000 },  
3   { "ngay": "2026-05-03", "soHoaDon": 8, "tongTien": 2780000 },  
4   { "ngay": "2026-05-04", "soHoaDon": 15, "tongTien": 5610000 }  
5 ]
```

Listing 4.17: Response mẫu của GET /api/hoadon/doanhthu?thang=5&nam=2026

4.4.5 Tiêu chuẩn RESTful

Web API tuân thủ các nguyên tắc REST:

- **Resource-based URL:** /api/benhnhan, /api/hoadon – không có verb trong URL.
- **HTTP Methods đúng ngữ nghĩa:** GET (đọc), POST (tạo), PUT (cập nhật), DELETE (xóa).
- **Status code chuẩn:** 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 404 Not Found.
- **Stateless:** mỗi request mang đủ thông tin, không lưu session.
- **Content-Type:** application/json (hỗ trợ thêm application/xml qua AddXmlSerializerFormatters)

4.4.6 Swagger UI

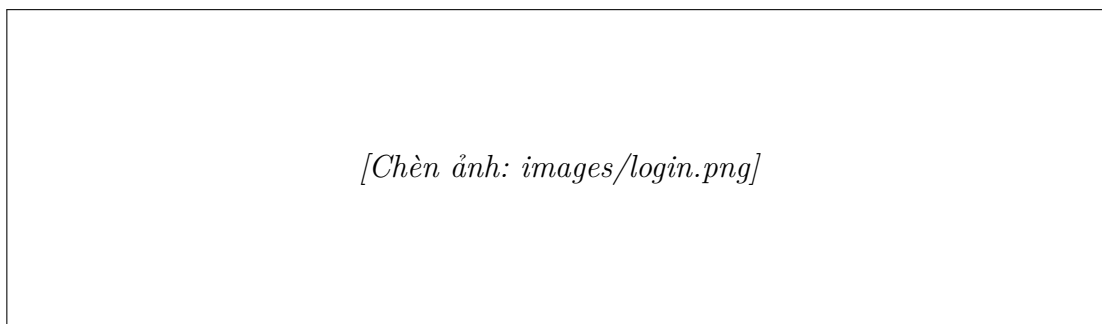
*[Chèn ảnh: images/swagger.png]
Swagger UI tại <https://localhost:7000/swagger>*

Hình 4.1: Tài liệu API tự động bằng Swagger UI

5. KẾT QUẢ VÀ ĐÁNH GIÁ

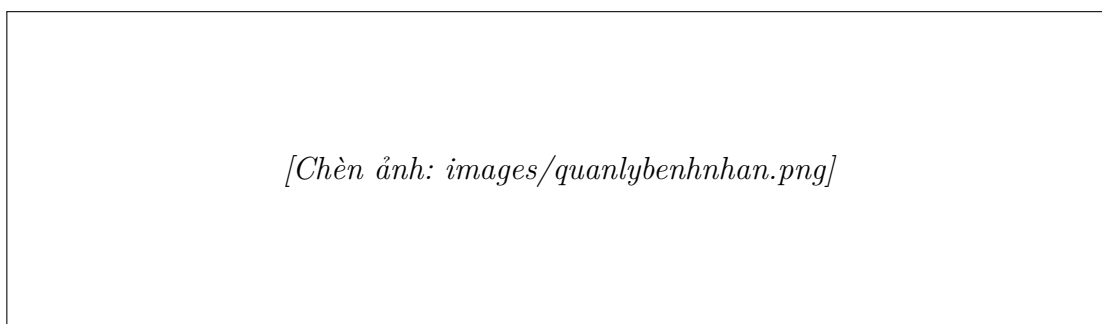
5.1 Demo ứng dụng

5.1.1 Đăng nhập



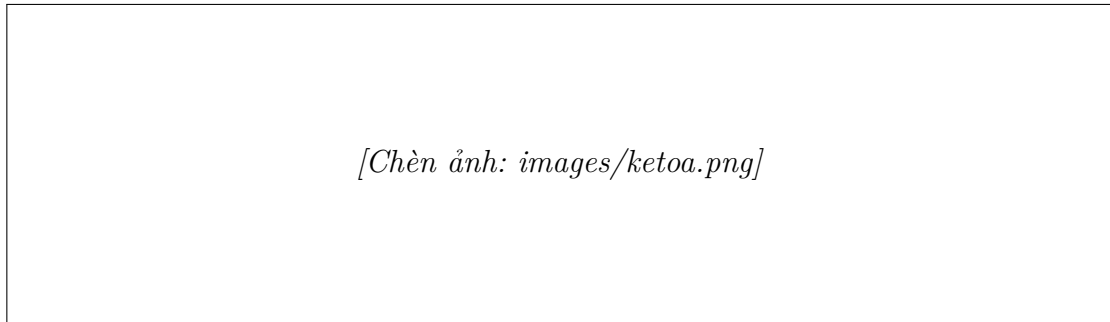
Hình 5.1: Đăng nhập – kiểm tra qua `sp_KiemTraDangNhap`.

5.1.2 Quản lý bệnh nhân



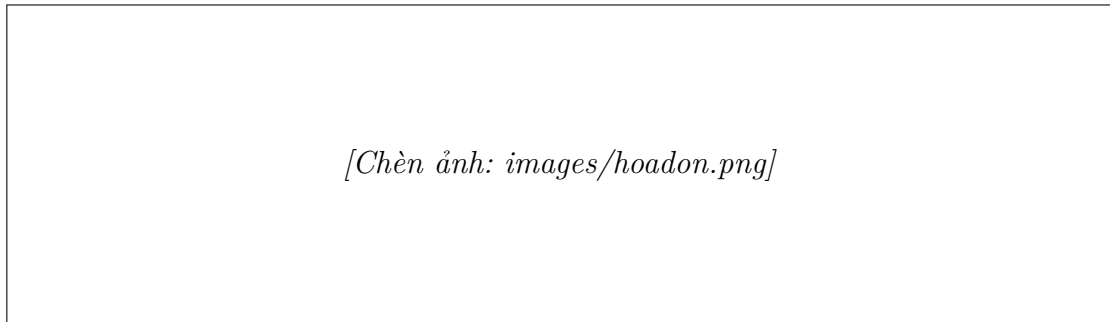
Hình 5.2: Quản lý bệnh nhân – thêm/sửa/xóa mềm/tìm kiếm.

5.1.3 Tạo phiếu khám và kê toa thuốc



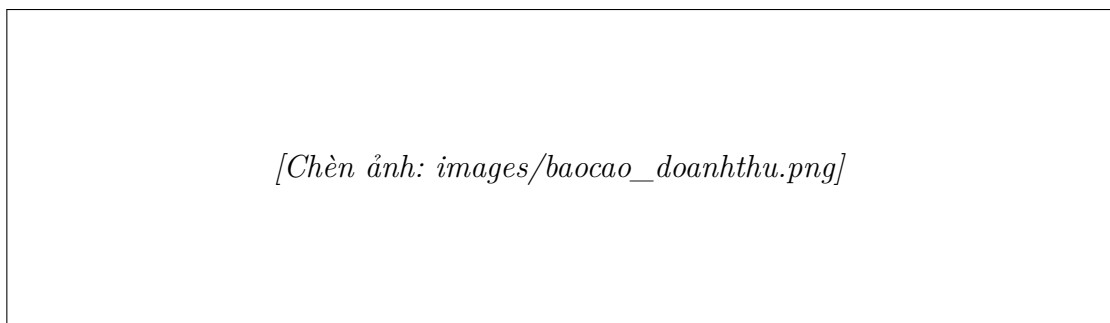
Hình 5.3: Kê toa – Trigger trừ kho tự động.

5.1.4 Lập hóa đơn và in PDF



Hình 5.4: Lập hóa đơn – SP có Transaction, xuất PDF iText7.

5.1.5 Báo cáo doanh thu



Hình 5.5: Báo cáo doanh thu tháng (DataSet + DataAdapter + LINQ).

5.2 Đánh giá chuẩn đầu ra

Bảng 5.1: Tự đánh giá mức độ đạt chuẩn đầu ra của môn học

Kỹ năng	Minh chứng trong đồ án	Mức đạt
T-SQL nâng cao	13 View, 25 Stored Procedure, 2 Function, 4 Trigger, 2 SP có Transaction.	Tốt
Thiết kế CSDL	15 bảng chuẩn 3NF, 17 FK, 3 CHECK, soft-delete.	Tốt
Kiến trúc 3 lớp + DTO	4 project độc lập, tách biệt rõ trách nhiệm.	Tốt
ADO.NET (kết nối)	14 lớp DAL với <code>SqlConnection/SqlCommand/SqlDataReader</code> .	Tốt
ADO.NET (phi kết nối)	<code>SqlDataAdapter + DataSet</code> cho các form báo cáo.	Tốt
LINQ to Objects	Lọc, sắp xếp, gom nhóm dữ liệu trong BUS/GUI.	Tốt
LINQ to XML	Xuất/nhập danh mục thuốc XML trong <code>LinqToXmlExamples</code> .	Tốt
Entity Framework Core 8	Code First (<code>DataAnnotations + Fluent API</code>) + Database First (<code>scaffold</code>), 14 <code>DbSet</code> , 2 Repository LINQ to Entities.	Tốt
Web API + RESTful + JSON	5 Controllers, 17 endpoints REST, Swagger UI, hỗ trợ JSON & XML.	Tốt
Kỹ năng làm việc nhóm	Phân chia rõ ràng, sử dụng Git/GitHub để cộng tác.	Tốt

5.3 Khó khăn và hướng phát triển

5.3.1 Khó khăn

- Đồng bộ schema CSDL giữa các thành viên ban đầu gặp trục trặc; nhóm khắc phục bằng cách tạo `script.sql` làm chuẩn và đẩy lên Git.
- Nghiệp vụ kê thuốc – trừ kho và lập hóa đơn yêu cầu Transaction và Trigger phức tạp,

để gây lỗi nếu thiếu `ROLLBACK` trong `CATCH`.

- Cú pháp `HasCheckConstraint` thay đổi giữa EF Core 6 và EF Core 7+, gây lỗi build khi nâng cấp; nhóm khắc phục bằng cách thống nhất dùng EF Core 8 và viết `ToTable(t => t.HasCheckConstraint(...))`.
- Mật khẩu hiện đang lưu plaintext trong bảng `TaiKhoan`; cần băm BCrypt trong giai đoạn nâng cấp bảo mật.
- Việc đồng bộ giữa WinForms client (gọi BUS/DAL trực tiếp) và Web API (EF Core) đòi hỏi định nghĩa rõ DTO; nhóm xử lý bằng cách tạo thư mục `GUI_QLPK/Services` làm tầng tích hợp với cơ chế fallback.

5.3.2 Hướng phát triển

- Bổ sung xác thực *JWT Bearer Token* cho Web API và phân quyền theo Role.
- Thay thế WinForms client bằng giao diện web (React/Blazor) gọi Web API qua HTTPS.
- Tích hợp thanh toán điện tử (VNPay/MoMo) và nhắc lịch qua Zalo/SMS.
- Áp dụng mã hóa mật khẩu (BCrypt) và HTTPS bắt buộc.
- Mở rộng `AuditLog` với Trigger ghi mọi thao tác Insert/Update/Delete quan trọng.
- Triển khai container hóa bằng Docker + CI/CD trên GitHub Actions.

TÀI LIỆU THAM KHẢO

1. Microsoft Learn – *Transact-SQL reference*. <https://learn.microsoft.com/en-us/sql/t-sql/>
2. Microsoft Learn – *ADO.NET overview*. <https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/>
3. Microsoft Learn – *LINQ*. <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>
4. Microsoft Learn – *LINQ to XML*. <https://learn.microsoft.com/en-us/dotnet/standard/linq/linq-xml-overview>
5. Microsoft Learn – *EF Core*. <https://learn.microsoft.com/en-us/ef/core/>
6. Microsoft Learn – *Web APIs with ASP.NET Core*. <https://learn.microsoft.com/en-us/aspnet/core/web-api/>
7. Microsoft Learn – *DML Triggers*. <https://learn.microsoft.com/en-us/sql/relational-databases/triggers/dml-triggers>
8. Microsoft Learn – *Stored Procedures*. <https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine>
9. Swashbuckle.AspNetCore – *Swagger for ASP.NET Core*. <https://github.com/domaindrivendev/Swashbuckle.AspNetCore>
10. Guna.UI2 – *Documentation*. <https://docs.gunaui.com/>
11. iText 7 – *Knowledge base*. <https://kb.itextpdf.com/>
12. Giáo trình *Lập trình cơ sở dữ liệu* – Khoa CNTT, Trường Đại học Mở TP.HCM.